SC23 ISO C++ Bof Panel

C++23 “Pandemic Edition” and C++ NEXT 26

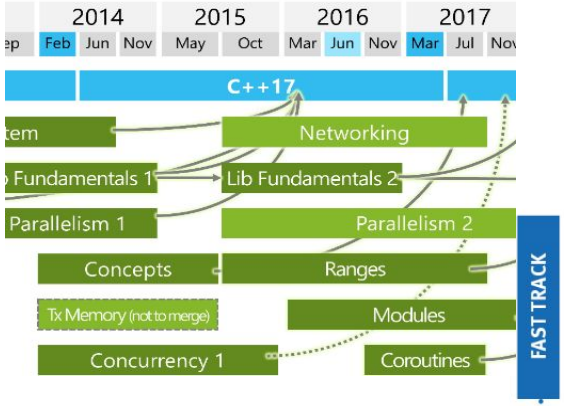
Michael Wong (Codeplay/Intel)
Gonzalo Brito (Nvidia)
Nevin Liber (ANL)
Weile Wei (LBL)
Christian Trott (Sandia)
Hal Finkel (DOE/SC)

Michael Wong (Codeplay/Intel)

DE, Chair of SG14 (Low Latency), SG19 (ML/AI), Directions Group, SG1, Concurrency TS editor, SG23 Safety

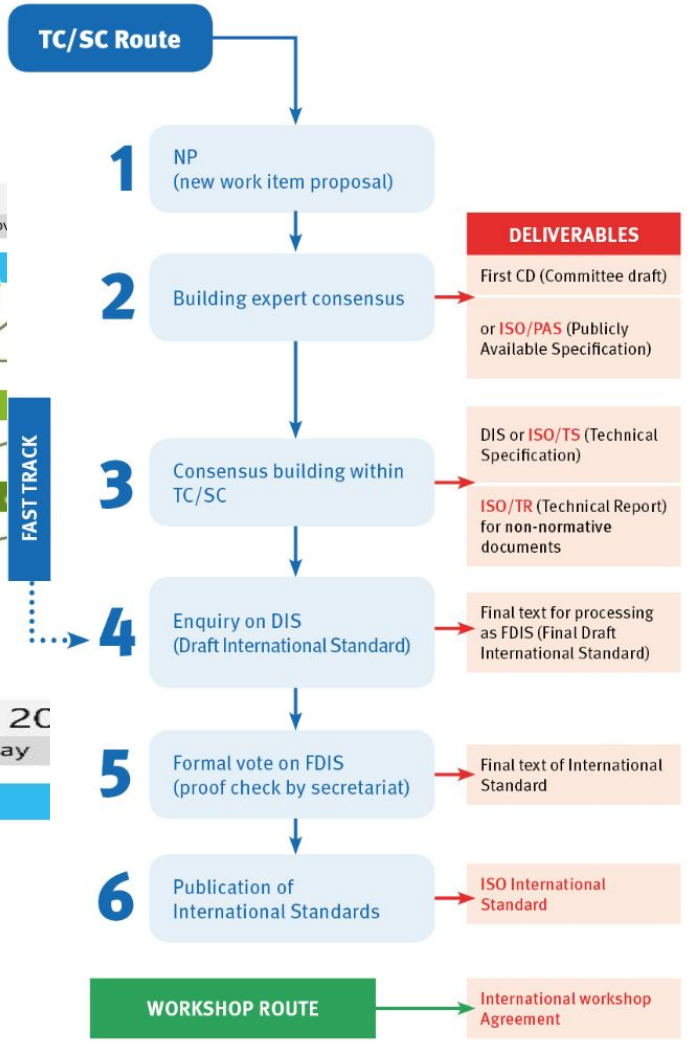
C++ 23 “Pandemic Edition”

- Concurrency TS1
  - improvements to std::future
  - Latches and barriers
  - Atomic smart pointers



- Concurrency TS2





What is in Concurrency TS2? It is on its way to Geneva!

- TS 2 is closed in 2023 in N4956 and is finally on its way to Geneva
  - 2 initial items on Deferred Reclamation - Hazard Pointers, RCU
    - https://github.com/cplusplus/concurrency-ts2
    - HP: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1121r3.pdf
    - RCU: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1122r4.pdf
  - 3 smaller items added in 2022 Kona and 2023 Issaquah meeting:
    - Asymmetric fences: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1202r5.pdf
    - Synchronized values: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0290r4.html
    - Byte wise atomics: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1478r8.html
  - *Then usually need a few more years for experience, so some could miss C++26*
- Great for Low latency: HP and RCU, Asymmetric Fence
- Great for Safety and convenience: Synchronized Values, Byte wise Atomics

Deferred Reclamation

Deferred Safe Reclamation

Protect dynamic objects from being reclaimed while in use.

Why?

Less synchronization: Low latency and low contention.

Read-Copy-Update (RCU) and Hazard pointer protection can **replace** slower less scalable **synchronization**

RCU (Folly library) ctor/dtor/protection **3.1** nanoseconds.

Hazard pointer (Folly) ctor/dtor/protection **3.9** nanoseconds.

Hazard pointer (Folly) pre-allocated protection **0.5** nanosecond.

Protection is practically **perfectly scalable**.

Safe Reclamation Techniques

- Read-write locks: std::shared_mutex
- Reference counting: std::atomic<std::shared_ptr>
- RCU
- Hazard pointers

Safe Reclamation Tradeoffs

- RW Locks and RCU protect many objects but are not suitable for long protection.
- Hazard pointers and reference counting are suitable for long protection but need to be specific about protected objects.
- Hazard pointers and RCU defer reclamation. RW-locks and reference counting reclaim immediately.

	RW Locks	Reference Counting	RCU	Hazard Pointers
Latency	High	High	Low	Low
Contention	Yes	Yes	No	No
Protection duration	Cannot be long	Can be long	Cannot be long	Can be long
Objects not yet reclaimed	Zero	Zero	Unbounded	Bounded
Irregular protection	General protection	Specific protection	General protection	Specific protection

Asymmetric Fences P1202

Asymmetric Fences P1202

Many synchronization algo have store-load fence, but the fence is required in a common path, only to guard against a rare race in an uncommon path

- HP: the tprotect and ry_protect() path is common, but the reclamation path it protects against is uncommon
- RCU: the rcu_reader constructor path is common, but the rcu_synhronize() path is uncommon
- Work-stealing and load-store mutexes: same problem

Problem for library authors

I have to put a store-load fence (the most costly kind) in a common paths, just to ensure safety against a race that is rare in a slow path

When I do that, it causes performance loss. Many systems go to great lengths on the fast-path to have a light-weight fence:

E.G. FB folly uses a membarrier() system call on the writer and only a compiler barrier on the reader

The following is 2 ns on a 2.5GHz Haswell with asymmetric barriers (from P1202R0)

- HP: hazptr_holder::protect and reset()
- RCU: rcu_reader construction and destruction

But 32 ns if we replace these with an atomic_thread_fence (memory_order_seq_cst)

Real workloads are even higher cost

Byte-wise atomic memcpy P1478

Byte-wise atomic memcpy P1478

Another mechanism to support low-cost read-mostly synchronizations by protecting shared data with an atomic sequence number.

Bytewise atomic allow individual byte atomics to be more efficient instead of making a large multibyte all atomic, which can cause inefficiency.

This allows values that may be read while being written when it can be determined after the fact that a race did not occur.

It also allows nonatomic accesses that behave like atomics and the most important use case is sequence locks.

Key Use case: seqlocks is a reader/writer lock that avoids writer starvation, writer increments sequence number (after acq lock and before rel lock), reader reads the seq number before and after reading shared data

If either are odd, writer had taken the lock while the data was being read and it may have changed; if numbers are different a writer has changed the data while it was being read. Either case, readers must retry in a loop until they read the same even sequence number (alternatively, the reader just gives up, similar to single-attempt trylock).

Synchronized Value P0290

Synchronized Value P0290

Synchronized values wrap a value with a mutex-like control to control access to the value, so that you don't forget about lock access and unlocking to facilitate race-free algo

`synchronized_value<T>` stores a value of type `T` and a mutex. It then exposes a pointer interface, such that derefencing the pointer yields a special wrapper type that holds a lock on the mutex, and that can be implicitly converted to `T` for reading, and which forwards any values assigned to the assignment operator of the underlying `T` for writing.

The `apply()` function then provides a means of accessing the stored value with the mutex locked, automatically unlocking the mutex afterwards.

The `apply()` function is variadic, so you can operate on a set of `synchronized_value<T>` objects. All the mutexes are locked prior to invoking the supplied callable object, and then they are all released afterwards.

The name is chosen to fit in with `std::apply` for tuples, since the operation is conceptually similar. Rather than expanding a `std::tuple` to supply the arguments to the function, the values wrapped by the `synchronized_value`s are extracted to supply the arguments to the function.

This provides an easy way for developers to ensure that all accesses to a given object are done with the relevant mutex locked, whilst also allowing for operations that require locks on multiple objects.

In order to avoid simple mistakes when using the `synchronized_value<T>` objects, there are no public member functions or operations other than construction.

The actual implementation may use an alternative synchronization mechanism instead of a mutex, provided that the synchronization requirements are met. Like spinlock

Gonzalo Brito (NVIDIA)

SG1 (Concurrency & Parallelism)

C++26: Improving C++20 Concurrency primitives

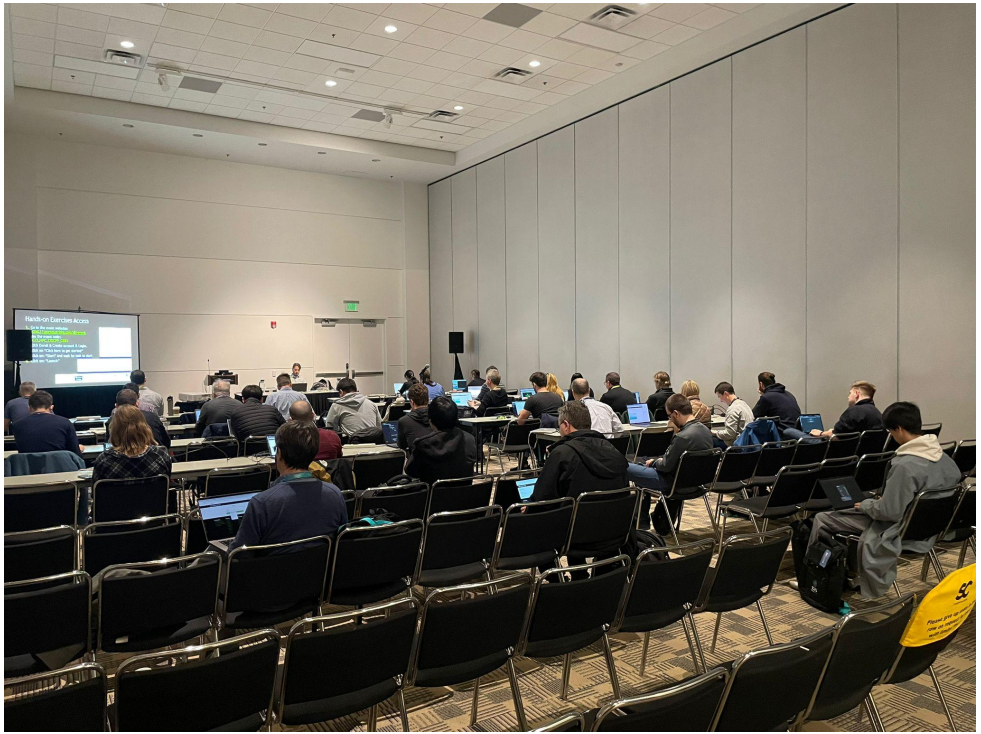
- atomic min/max (P0493 and P3008)
- atomic_ref contention detection (P2835)
- Many improvements to “wait” (P2643):
  - Return last observed value:
    atomic<>::wait_value(old, mo) -> T
  - Timed waits: try_wait, try_wait_for, try_wait_until
    For atomic, barrier, and latch
  - Predicated waits: try_wait(stop_predicate, mo)
  - Make atomic wait usable in more situations (P2616)

C++17 Parallel Algorithms are great!

Program heterogeneous HPC systems in portable ISO C++!

Supported by all major CPU and GPU HPC vendors!

The SC ‘23 tutorial peaked at 35 attendees! ~30% growth!



C++17 Parallel Algorithms Two Main Limitations

- **Synchronous (block the calling thread):**
  little parallel work → high sync overhead → low utilization

- **Execution resource selection:**
  cannot select on which device individual algorithms run

ISO C++ Asynchrony

Structured concurrency framework

Task Graphs + Execution Contexts

- **Portably describe** complex asynchronous task graphs.
- **Dispatch** them to standard or vendor schedulers.

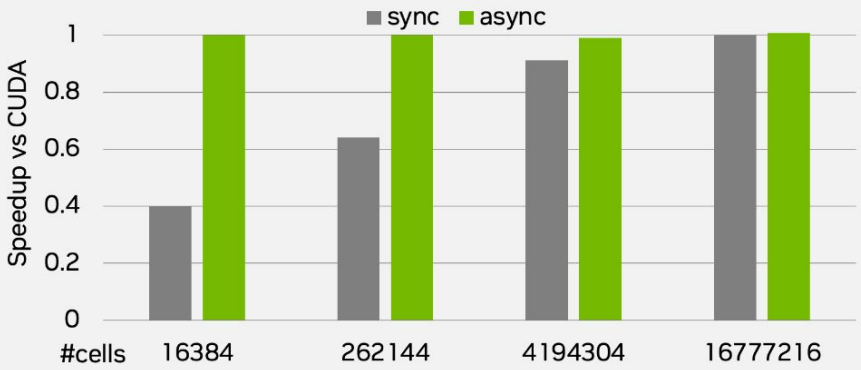
```
// Execution contex: expensive (threadpool, stream)
auto ctx = static_thread_pool(5);

// Cheap handle into context:
stde::scheduler auto sch = ctx.get_scheduler();

// Lazy task graph creation
auto graph = stde::schedule(sch)
  | std::bulk(n, [&](int i) { … })
  | std::bulk(n, [&](int i) { … });


// Start task graph and block current thread:
std::sync_wait(graph);
```

Performance parity with CUDA C++



Synchronous ISO C++ parallel algorithms

```
for (int step = 0; step < n_steps; step++) {
  std::for_each(stde::par, …);
  std::for_each(stde::par, …);
}
```

Asynchronous CUDA using Streams

```
for (int step = 0; step < n_steps; step++) {
  kernel<<<blocks, threads, 0, stream>>>(…);
  kernel<<<blocks, threads, 0, stream>>>(…);
}
cudaStreamSynchronize(stream);
```

Weile Wei (LBL)

Unified, portable, standard C++ code on CPU & GPU

C++17 parallel STL – nvc++ stdpar

```
std::for_each_n(
      stdex::par,//[par_unseq|seq]
      counting_itr(0),
      size,  [=] (std::size_t i) {
            func(i);
});
```

C++26 std::execution – Nvidia's stdexec

```
ex::sender auto res = ex::just()
      | exec::on(sch, //[cpu|gpu|multi-gpu]
            repeat_n(n,
                  ex::bulk(size, [=](int i) {
                        func(i);})
      | ex::then( [=] () { }) ) );

ex::sync_wait(std::move(res));
```

Run same code on CPU | GPU | (multi-GPU) with one line change.

Unified, portable, standard C++ code on CPU & GPU

Apps:

1D & ***2D stencils*** (std::mdspan),
FFT
task-based Cholesky (std::linalg)

Code:

NERSC hpcpp apps:
https://github.com/NERSC/hpcpp
NVIDIA's stdexec:
https://github.com/NVIDIA/stdexec

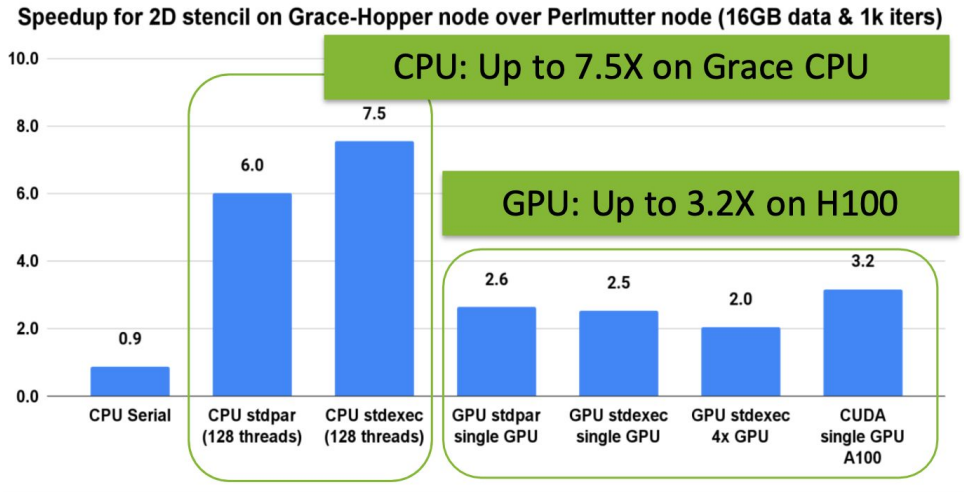


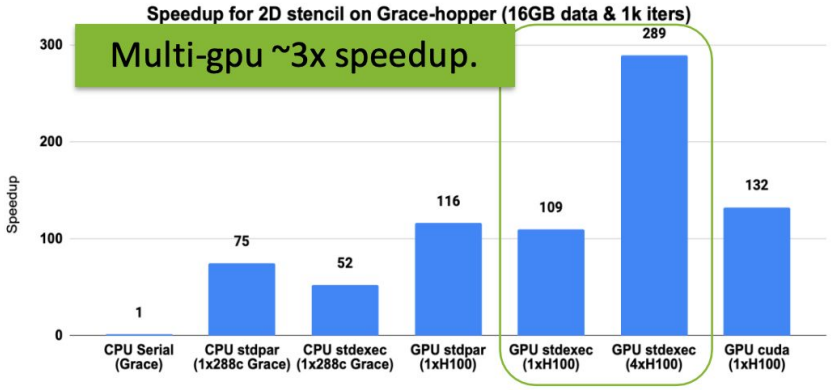
Experiment setup:

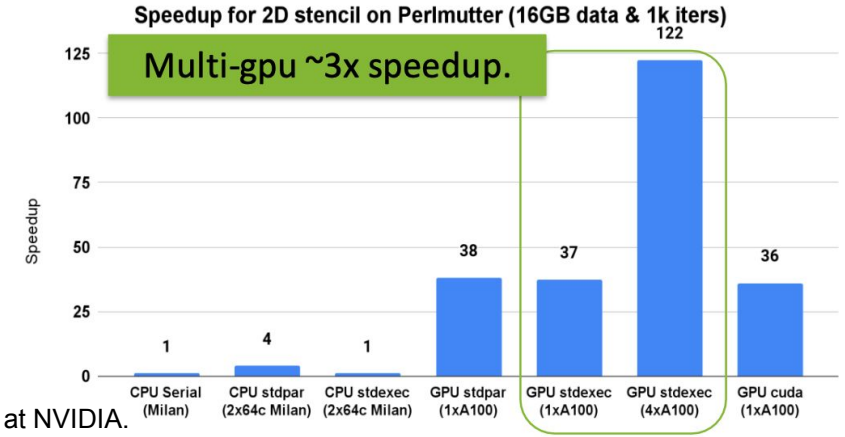
Spec	Perlmutter (CPU)	Perlmutter (GPU)	*Grace-hopper **Pre-Production**
CPU & GPU	2	1	4
CPU sockets	64	64	288
CPU mem/socket	256GB	256GB	120GB
NUMA nodes	8	4	4
CPU mem/NUMA node	64GB	64GB	[0-3] 120GB
GPUs per node	0	4	4
GPU memory	N/A	40GB	80GB
Compilers	nvhpc/23.7. gcc/12.2.0	nvhpc/23.7, gcc/12.2.0	nvhpc/23.5, gcc/12.3.0

*Grace-hopper node used here is a **pre-production** testbed at NVIDIA.

Unified, portable, standard C++ code on CPU & GPU







Feel free to reach out!

Weile Wei: WeileWei@lbl.gov

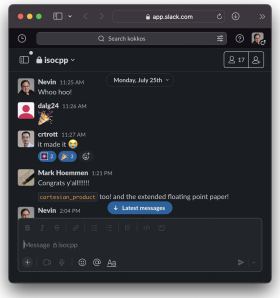
Muhammad Haseeb: mhaseeb@lbl.gov

*Grace-hopper node used here is a **pre-production** testbed at NVIDIA.

Nevin Liber (ANL)

mdspan - C++23 *non-owning* multidimensional array view

```
template<class ElementType,
         class Extents,
         class LayoutPolicy = layout_right,
         class AccessorPolicy = default_accessor<ElementType>>
struct mdspan {
    template<class... OtherIndexTypes>
    constexpr reference operator[](OtherIndexTypes... indices) const;
    // ...
};
```



`operator[]` - C++23 variadic (and/or `static`)

```
template<class ElementType,
         class Extents,
         class LayoutPolicy = layout_right,
         class AccessorPolicy = default_accessor<ElementType>>
struct mdspan {
    template<class... OtherIndexTypes>
    constexpr reference operator[](OtherIndexTypes... indices) const;
    // ...
};
```

`extents` - C++23 multidimensional index space

```
template<class ElementType, size_t Extent>
struct span {
    using size_type = size_t;    // I'm sorry
    // ...
};

template<class IndexType, size_t... Extents>
struct extents { /* ... */ };

template<class ElementType, class Extents, class LayoutPolicy, class AccessorPolicy>
struct mdspan {
    using size_type = make_unsigned<Extents::IndexType>;
    // ...
};

template<class IndexType, size_t Rank>
using dextents = /* ... */;
```

C++26 ***mdspan***-related Improvements *(in-flight)*

- P2630 **`submdspan`** *(adopted Varna 2023-June)*
  - The subspan (“slicing”) function that returns a subset of an existing `mdspan`

- P2642 padded `mdspan` layouts
- P2689 **`atomic_accessor`**
- P2897 **`aligned_accessor`**: `mdspan` accessor for pointer overalignment

- P1684 **`mdarray`**: *Owning* multidimensional array analog of `mdspan`
  - Container adaptor (`queue`, `priority_queue`, `stack`, `flat_map`, `flat_multimap`, `flat_set`, `flat_multiset`)

C++26 Library Improvements *(in-flight)*



- P2819 Add `tuple` protocol to `complex`
  - Structured bindings support
  - *Adopted 4 days ago in Kona*
  - Will be added to *`Kokkos::Complex`* & *(proposed)* *`sycl::complex`*

- P1928 **`std::simd`** - Merge data-parallel types from the Parallelism TS 2
- P0843 **`inplace_vector`**
  - A dynamically-resizable vector with fixed capacity and embedded storage



- Networking

C++26 Safety Improvements *(in-flight)*

- P2795 Erroneous Behavior for Uninitialized Reads
  - **Automatic (stack) variables are fully initialized**
    - Including padding
    - It's value cannot changed until assigned-to
  - Reading an uninitialized variable is definitely a bug
  - Conforming compilers still have to accept it
    - **Well-defined behavior, not *undefined behavior***
  - Can reject in non-conforming modes
  - Principles on applying it to undefined behavior

C++26 Safety Improvements *(in-flight)*

- P2759 DG Opinion of Safety for ISO C++
  - Profiles (Bjarne Stroustrup)
    - Collection of restrictions, requirements, related semantics that define a safety property to be enforced
      - Specifically, not a subset of C++
      - E.g., a range-safety profile can't ban unchecked subscripting, but instead provides runtime checked alternative
    - **Do not change semantics of a valid program**
      - With the exception of undefined behavior <==> well-defined behavior
    - Open (hard) questions: how do different profiles in the same code base work together?
    - P3038 Concrete suggestions for initial Profiles

C++26 Safety Improvements *(in-flight)*

- P2900 Contracts for C++
  - The ability to add annotations to and within functions that will be used to identify when incorrect behavior has occurred
    - May or may not result in a runtime check
  - More knobs for dealing with (library) undefined behavior
  - MVP (Minimum Viable Product)
    - CCA (Contract-Checking Annotation)
      - Preconditions, postconditions, assertions
      - Semantic
        - Ignored, enforced, observed
          - Still ***undefined behavior*** if not enforced (terminated)

C++26 Language Improvements *(in-flight)*

- P2996 Reflection for C++26
  - Static reflection
  - EWG (Evolution Working Group) will focus on reflection at next meeting (Tokyo 2024-March)

2023-11 Kona ISO C++ Committee Trip Report

- Second C++26 meeting!
  - Last week
  - Published yesterday

https://www.reddit.com/r/cpp/comments/17vnfqg/202311_kona_iso_c_committee_trip_report_second/







Christian Trott (Sandia)

Linear Algebra comes to ISO C++ 26

Design Principles:

Start with the performance critical basis for all linear algebra: BLAS

- *Propose equivalent of every function in BLAS*

Linear Algebra Functions are just another set of C++ algorithms like sort

- *Propose a free function interface similar to standard algorithms*

Encapsulate data in the minimal fundamental representation of multi dimensional arrays still providing necessary flexibility

- *Use `mdspan` as parameters*

Ensure that new use cases such as Machine Learning are covered

- *Turns out mdspan can give that to us naturally*

Voted into the draft 2023-11-11

Matrix-Vector Product Punchline

```
matrix_vector_product(A, x, y);
```

Matrix-Vector Product Example

Standard usage, just create the mdspans and hand them to C++ std::linalg

```
double * A_ptr = …, *x_ptr = …, *y_ptr = …;
mdspan A(A_ptr, nRows, nCols); // C++17 constructor template arg deduction
mdspan x(x_ptr, nCols);
mdspan y(y_ptr, nRows);

matrix_vector_product(A, x, y);
```

More advanced: mixed precision, transpose and scaled

```
float * A_ptr = …; double *x_ptr = …, *y_ptr = …; // mixed precision
mdspan A(A_ptr,nRows,nCols); mdspan x(x_ptr,nCols); mdspan y(y_ptr,nRows);

matrix_vector_product(
  std::execution::par,    // execution policy argument to ask for par
  transpose(A),           // returns new mdspan with transposed layout
  scaled(x, 2),           // returns new mdspan with accessor for scaling
  y);                     // using double accumulation since y is double
```

std::linalg Capabilities

- All BLAS algorithms (plus a couple extras)
  - No LAPACK though
- Mixed precision
  - Mix 64bit, 32bit, 16bit, …
- Mix of compile and runtime extents
  - This should be good for doing 4x4 matrices
- Memory layout awareness
  - C-Layout, Fortran-layout, strides in both directions, tiled layouts, …
- User defined scalar types
  - e.g. fixed-point real numbers or UQ types
- Allows maximal implementation freedom